

[Short Paper] Which Neighbors Count? Query-Time Subspace Search and In-Database Attribution in XQdrant

Moetez Fradi
moetez.fradi@insat.ucar.tn
INSAT
Tunis, Tunisia

Abstract

Vector databases return opaque scalar similarities over high-dimensional embeddings, giving practitioners no coordinate-level explanation of a match and no way to restrict HNSW traversal itself to a dynamic focus subspace. We present **XQdrant**, an extension of the Qdrant engine that addresses both.

Attribution: with `dims_explained` computes exact top- m per-dimension score decomposition in-database, avoiding client-side vector egress. It leaves the ranked result unchanged, adds about 0.3 ms of in-engine time independent of m , and runs $\sim 1.8\times$ faster end to end than post-query extraction at encoder-scale widths ($D \in \{768, 1536\}$).

Masked-distance HNSW traversal scores only the focus subspace during descent over an unmodified full-space graph, and evaluating it requires two ground truths. Against full-space ground truth, masked search on SIFT1M matches the subspace/full-space k -NN coherence (0.012 vs. 0.013 at $D_{\text{sub}}/D = 0.1$), a property of the data rather than a navigation failure. Against subspace ground truth, it finds the focus-metric neighbors with Recall@10 of at least 0.91 when the focus set covers half the dimensions or more, but 0.77 at a quarter and about 0.4 at a tenth; keeping full-distance routing on the upper layers (`mask_from_layer`) changes this by at most 0.04. In-engine, masked search runs at $0.67\text{--}0.85\times$ the speed of plain search. With every dimension in focus it returns identical results yet takes $1.6\times$ as long, and a masked distance over a quarter of the dimensions costs twice a full contiguous one, so per-distance cost, not traversal, rules out a latency win. Overfetch plus full-distance verification recovers full-space recall where coherence allows ($0.48 \rightarrow 0.91$ at $D_{\text{sub}}/D = 0.75$) for under 0.3 ms, and an α -weighted full/masked blend never improves on pure masking. We release the harness and an experiment index that maps each claim to a script.

Keywords

vector databases, HNSW, approximate nearest neighbor, score attribution, query explanation, subspace search, masked distance, SIMD

1 Introduction

High-dimensional dense embeddings underpin retrieval-augmented generation (RAG), recommendation, and semantic search [14, 22, 25]. Production engines traverse HNSW graphs [17], collapsing every comparison into a single scalar. This creates two gaps that motivate this paper.

Explaining scores. Practitioners cannot see which coordinates drove a hit without fetching full vectors and recomputing per-dimension terms client-side, an expensive, egress-bound

pattern. Per-coordinate terms matter most when coordinates carry meaning, as in concatenated multi-field or multimodal vectors [26], where they sum to each field’s share of the score.

Subspace routing. Many workloads care about a dynamic focus set $\mathcal{F}$ of coordinates: one field or modality of a concatenated vector, a Matryoshka prefix that is itself a lower-dimensional embedding [13], or the dimensions under study in an ablation. Yet stock HNSW always evaluates full D -dimensional distances during graph descent. Rescoring on $\mathcal{F}$ after full-vector preselection improves semantic specificity but, as we show, cannot reduce traversal cost.

XQdrant addresses the first gap and characterizes the second. For attribution, with `dims_explained` attaches exact top- m coordinate terms to search results in-database. For subspace routing, `nearest.focus` with `masked=true` evaluates reduced distances during HNSW traversal, not just after it, with a hybrid layer cutoff, a choice of masked-distance kernels, an optional verify/overfetch pass, and a full/masked score blend. Evaluating masked traversal requires care about which neighbors count as correct. Recall against full-space ground truth measures whether two metrics agree; recall against subspace ground truth measures whether traversal navigates. Keeping the two apart changes the conclusions (Section 5.3).

Contributions

- **In-database attribution.** Exact score decomposition with order-statistic top- m selection; about 0.3 ms in-engine and $\sim 1.8\times$ faster than post-query extraction, with the ranked result unchanged.
- **Masked-distance HNSW traversal, evaluated under two ground truths.** On SIFT1M it navigates well for large focus sets and degrades for small ones, a layer cutoff barely changes this, and it never beats plain search; we trace the latency gap to per-distance cost, which a microbenchmark puts at twice a full distance for a quarter of the dimensions.
- **Coherence as the reference for full-space recall.** Subspace/full-space k -NN overlap matches the full-space recall of masked search and predicts when a verify pass can recover full-space neighbors, independent of the traversal strategy.
- **Reproducible artifacts.** A public experiment index and harness (Section 8) mapping every claim above to a canonical script and dataset.

2 Background

2.1 HNSW and Contiguous SIMD

HNSW maintains multi-layer proximity graphs [17]. Queries greedily descend with candidate heap size `ef_search`, then extract top- k at the base layer. Distance kernels assume contiguous $f32$ slices (AVX2/NEON), so a non-contiguous focus set must

either (i) repack selected coordinates into a scratch buffer before scoring, or (ii) gather them directly via SIMD gather loads.

2.2 Score Decomposition

For Dot product, $s(\mathbf{q}, \mathbf{v}) = \sum_i q_i v_i$; Cosine applies the same decomposition after ℓ_2 normalization; Euclidean and Manhattan admit analogous additive terms. Attribution is therefore exact accounting of the engine's own metric, not an estimate (Section 6).

2.3 Subspace Coherence

Let $N_k^{\text{full}}(q)$ and $N_k^{\mathcal{F}}(q)$ denote exact k -NN sets under the full metric and under focus set $\mathcal{F}$. We define *coherence* as $|N_k^{\text{full}} \cap N_k^{\mathcal{F}}|/k$ and call this diagnostic **C1**. Coherence is the full-space Recall@ k that an *exact* subspace search achieves; it depends on the data and $\mathcal{F}$, not on any index. Masked traversal thus has two accuracy measures. *Subspace recall*, against $N_k^{\mathcal{F}}$, asks whether traversal found the focus-metric neighbors, i.e., whether the graph remains navigable. *Full-space recall*, against N_k^{full} , asks whether those neighbors are also full-space neighbors; for a search that returns $N_k^{\mathcal{F}}$ exactly, it equals coherence.

3 In-Database Attribution

XQdrant injects attribution at the collection coordinator after shard merge, with an early return when `with_dims_explained` is absent, so other requests take the unmodified path. Per explained hit, Algorithm 1 materializes D terms and selects the top- m by absolute contribution in average $\mathcal{O}(D)$ time via Rust's order-statistic selection (`select_nth_unstable`).

Algorithm 1 (In-database top- m attribution).

Input: query $\mathbf{q}$, vector $\mathbf{v}$, D , m , metric δ .

1. Normalize operands if required by δ .
2. For $i = 0, \dots, D - 1$: append $(i, \text{TERM}_i(\mathbf{q}, \mathbf{v}, \delta))$.
3. Return order-statistic top- m by $|t|$, sorted descending.

Figure 1: In-database top- m feature attribution.

nearest.focus without masking still performs full HNSW preselection and rescores on $\mathcal{F}$ afterward, useful when ranking must respect a focus subspace, but, as Section 5.2 shows, not a traversal-cost reduction. That gap is exactly what motivates masked traversal (Section 4).

4 Masked-Distance HNSW: Design Space

We consider three traversal-level strategies for evaluating distance only over a focus subspace $\mathcal{F}$ during HNSW descent, plus two supporting mechanisms (a kernel choice and a recall-recovery pass) that apply across all three. Table 1 fixes the naming used for the rest of the paper.

4.1 API Surface

Focus parameters ride inside the existing query body:

```
"focus": {
  "dims": [...], "masked": true,
  "mask_from_layer": 1, // M2/M3 hybrid cutoff
  "verify": true,       // optional full rescore
  "alpha": 0.25         // M3 blend weight
}
```

Constraints mirror existing dense local-search paths: no quantization/turbo, no simultaneous `with_dims_explained` on masked

Table 1: Naming used throughout Sections 4–5.

Name	What it is
M1	Naive: mask every HNSW layer
M2	Hybrid: full distance for layers $\geq L$, masked below (<code>mask_from_layer = L</code>)
M3	M2 plus an α -weighted blend of full/masked scores on masked layers
K1	Kernel choice: repack-then-score vs. SIMD gather, orthogonal to M1–M3
V1	Verify/overfetch: optional post-traversal full-distance rescore, orthogonal to M1–M3
C1	Coherence diagnostic (Section 2.3), used to interpret M1–M3 and V1

paths, and fail-fast if `mask_from_layer`, `verify`, or `alpha` are set without `masked=true`.

4.2 M1: Naive All-Layer Masking

With `masked=true` and no cutoff, every HNSW comparison during descent uses only coordinates in $\mathcal{F}$. This is the simplest semantic change and the natural first attempt. The concern is navigability: graph edges encode full-space proximity, so greedy descent under a different metric has no guarantee of reaching the focus-metric neighbors. Section 5.3 tests this directly.

4.3 M2: Hybrid Layer Cutoff

M2 keeps full-distance routing where the graph is coarsest and masks only near the base layer, where the final candidates are chosen. Layer ℓ uses the masked scorer iff $\ell < L$ for cutoff $L = \text{mask_from_layer}$; otherwise full distance is used. $L=0$ recovers plain (unmasked) nearest search; a large L recovers M1. The implementation builds dual raw scorers inside `FilteredScorer` and switches via `set_layer` at layer boundaries; visited-set and heap logic are unchanged, so this composes with the rest of the query pipeline for free.

4.4 M3: Weighted Blend

On masked layers, $\text{score} = \alpha \cdot s_{\text{full}} + (1 - \alpha) \cdot s_{\text{focus}}$; coarse (unmasked) layers are unaffected. Note that blending computes *both* distances, so it cannot reduce arithmetic cost on its own; it exists purely as a routing-bias ablation to test whether it recovers recall at a fixed cutoff more cheaply than raising L .

4.5 K1: Kernel Choice: Repack vs. Gather

Masked scoring can copy focus dims into a contiguous scratch buffer before calling the existing SIMD kernels (*repack*, the default), or gather them directly in-register (AVX2 gather for Dot/Cosine/Euclidean on x86; scalar fallback elsewhere). We add the gather path, selectable via environment variable `XQDRANT_MASKED_KERNEL`, and treat kernel choice as orthogonal to M1–M3: any of the three can use either kernel.

4.6 V1: Verify and Overfetch

`verify=true` re-scores returned candidates with full distance after traversal, reusing the existing quantization-rescore hook. At overfetch $m=1$ (i.e., *limit* = k) this only reorders a size- k set, so Recall@ k is provably unchanged; it is a correctness/ordering feature, not a recall-recovery one. Overfetch with $m > 1$ (*limit* = $m \cdot k$, keep top- k after full rescore) turns masked traversal into

a candidate *filter* and can recover recall, at a latency cost we quantify in Section 5.6.

5 Experimental Evaluation

Table 4 at the end of this section summarizes the verdict for each mechanism.

5.1 Setup

Hosts and protocol. Host A (Apple MacBook Air, M2, 8 cores, 16 GB RAM, macOS) runs only the client-observed attribution suite (Figure 2 and the client-observed rescoring speeds in Section 5.2) against vanilla Qdrant (1.18.3-dev, commit b4e9326); no latency is compared across hosts. Host B (AMD Ryzen 5 220, 6 cores/12 threads, 15 GB RAM, AVX2) runs every masked-traversal experiment, all in-engine timings, and the kernel microbenchmarks, with XQdrant at commit ed5ca79 (opt-level 3, no LTO) in WSL2 Ubuntu 24.04; its plain-search baseline is XQdrant without focus parameters, which takes the unmodified Qdrant path. Collections use HNSW with $M=16$ and $ef_construct=200$, no quantization, and, unless swept, $ef=128$; the masked kernel is repack, the engine default, unless the kernel is the subject (it changes how a score is computed, not what is computed, so recall does not depend on it). All masked-traversal comparisons on a corpus share one index, so differences between mechanisms are not confounded by index construction. Unless a caption says otherwise, each configuration is the mean of $T=5$ trials with distinct seeds that redraw the focus set and, where applicable, the query sample, with sample standard deviation as error bars. A focus set at ratio r is a uniformly random subset of round(rD) coordinates. In-engine latency is Qdrant’s server-side request time, which excludes HTTP and serialization; client round trips over keep-alive connections can add a ~ 40 ms TCP delayed-ACK stall for multi-packet messages, which would swamp the sub-millisecond effects we measure.

Attribution suite. Synthetic corpora with $N=10k$ vectors and $Q=500$ queries at $D \in \{768, 1536\}$; baselines are vanilla Qdrant and post-query extraction, which fetches the result vectors (with_vector) and computes the top- m terms in the client.

Masked-traversal suite. SIFT1M [11] ($D=128$, Euclidean, $N=1M$, $Q=500$, $k=10$), on which plain HNSW search reaches Recall@10 of 0.99, and a synthetic corpus with $N=50k$ at $D=768$: i.i.d. standard-normal vectors, ℓ_2 -normalized, scored by Dot product.

5.2 Attribution and Focus Rescoring Are Not the Same Problem

Attribution annotates an already-ranked result, so it cannot change recall: at $D=768$, Recall@10 is identical for vanilla Qdrant and for XQdrant with $m=10$ attributions at every ef we tested. In-engine (host B), it adds 0.29–0.32 ms to a 2.12 ms plain search, about 14%, and the cost barely depends on m (2.42, 2.41, and 2.44 ms for $m = 1, 10, 50$). The relevant comparison is post-query extraction, the only alternative available to a client: Figure 2 shows in-database attribution holds a flat mean p50 of 3.85–4.02 ms (± 0.07) across $m \in \{1, \dots, 50\}$ while post-query extraction sits at 6.87–7.03 ms ($\approx 1.77\times$). These client-observed timings are free of the delayed-ACK stall noted above: the harness sends both multi-packet requests, the attribution query and the vector fetch, on fresh connections, and every p50 is far below 40 ms. The gap is what extraction adds: a second round trip, vector

egress, and client-side computation. nearest. focus *rescoring*, which reuses full-vector preselection, is slower than plain search: $0.80\text{--}0.84\times$ its speed client-observed at $D=768$ over ratios 0.1–0.75 (host A), and $0.80\text{--}0.81\times$ in-engine on SIFT1M (Table 3). It reduces the dimensionality of the arithmetic but not the traversal it depends on. This motivates masking the traversal itself.

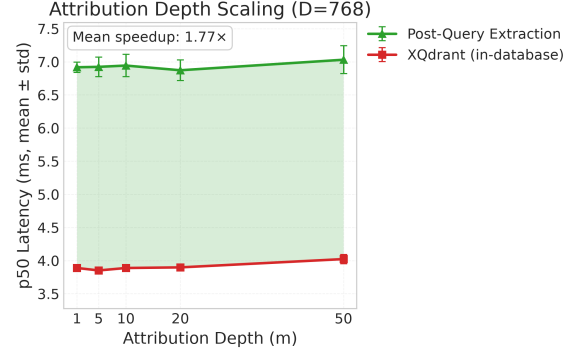


Figure 2: Attribution depth: in-DB vs. post-query extraction ($D=768$, host A, client-observed). Mean $\pm$ std over $T=5$ trials.

5.3 M1 and C1: Two Questions, Two Ground Truths

If rescoring cannot help, the next step is to mask distance *during* traversal (M1). Figure 3 evaluates M1 on SIFT1M against both ground truths, together with exact coherence. Against full-space ground truth, Recall@10 falls from 0.99 at ratio 1.0 to 0.490, 0.253, 0.078, and 0.012 at ratios 0.75, 0.5, 0.25, and 0.1; read alone, this looks like a navigability collapse. But coherence, computed without any index, matches it at every ratio (0.490, 0.250, 0.075, 0.013): M1 scores what a correct subspace search would, and the full-space numbers measure disagreement between the two metrics, not a failure to navigate. Against subspace ground truth, M1 finds the focus-metric neighbors with Recall@10 of 0.98 and 0.93 at ratios 0.75 and 0.5, but only 0.77 at 0.25 and 0.38 at 0.1. Navigability therefore holds for large focus sets and degrades for small ones, a much milder effect than the full-space numbers suggest.

5.4 M2: The Cutoff Barely Matters

Does keeping full-distance routing on the upper layers recover the subspace neighbors that M1 misses? Table 2 compares cutoffs on the same SIFT1M index. It barely matters: $L=6$ reproduces M1 exactly, cutoffs $L=2\text{--}4$ gain 0.04 at ratio 0.1 (0.42 vs. 0.38, within the across-trial spread), and no cutoff gains more than 0.01 at larger ratios. Masking only the base layer ($L=1$) already loses as much as masking every layer, so the loss arises where both designs mask: the base-layer search under the focus metric. A larger ef for masked queries is the natural remedy, which we did not sweep.

The synthetic $D=768$ corpus cannot inform this question. Plain full-space search on it reaches only 0.19 Recall@10 at $ef=128$, and masked search reaches 0.21 subspace recall for every $L \geq 1$ and every ratio we tested (0.25–0.75): masking costs nothing there because the index is already lost without it. Its coherence is also lower than SIFT’s (0.006 at ratio 0.1 to 0.30 at 0.75).

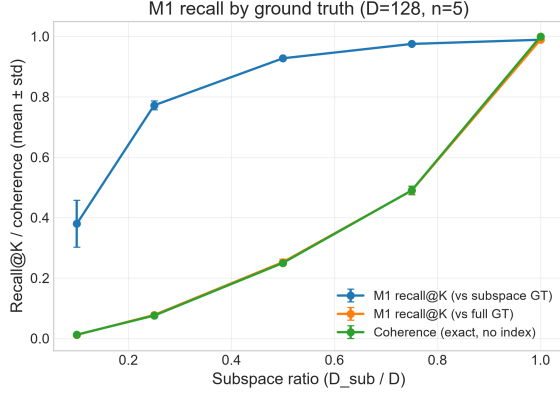


Figure 3: M1 on SIFT1M: Recall@10 against subspace and full-space ground truth, and exact coherence (no index), vs. ratio (mean±std, $T=5$). The full-space curve lies on the coherence curve.

Table 2: Subspace Recall@10 on SIFT1M by cutoff, one index, $T=5$. Across-trial std is ≤ 0.02 , except 0.05–0.08 at ratio 0.1.

Design	ratio 0.1	0.25	0.5
M1 (all layers masked)	0.38	0.77	0.93
M2, $L=1$	0.42	0.77	0.91
M2, $L=2-4$	0.42	0.78	0.93
M2, $L=5$	0.40	0.77	0.93
M2, $L=6$	0.38	0.77	0.93

5.5 Latency: Per-Distance Cost Rules Out a Win

No masked configuration beats plain search (Table 3). In-engine, masked search runs at 0.84–0.85× the speed of plain search at ratio 0.25 and at 0.67× at ratio 0.75, so it gets slower as the focus set grows. A masked request with $L=0$ runs at full speed, so the request path itself adds nothing. With every dimension in focus (ratio 1.0), masked search returns the same result set as plain search for all 500 queries, so it traverses the same nodes, yet it runs at 0.62× the speed of plain search (1.67 vs. 1.04 ms for M1 in a separate interleaved run; 0.64× at $L=1$). The gap is therefore per-distance cost, which Figure 4 measures in isolation: one contiguous 128-d Euclidean distance takes 18 ns, while a masked distance takes 36 ns over only 32 coordinates and 82–94 ns over all 128, whichever kernel computes it. Masking a quarter of the dimensions thus still doubles the cost of every distance. Gather does not help: in isolation it ties repack at 32 coordinates and is 15–20% slower at 96–128, and in-engine the two kernels are within 3% of each other (M1 at 0.84× vs. 0.82× of plain speed at ratio 0.25, 0.67× vs. 0.65× at 0.75). For masking to pay off, a masked distance must cost less than a full contiguous one. Laying registered focus dimensions out contiguously, as a Matryoshka prefix or a per-field block already is, gets there (14 vs. 18 ns at a quarter of the dimensions), though at SIFT width the saving is small; cheaper approximate distance comparisons [3, 6, 7] are the other route.

Table 3: In-engine latency on SIFT1M (host B, repack kernel): server-side p50 in ms over 500 queries (best of 3 per query, all rows interleaved per query) and speed relative to plain search.

Query	ratio 0.25	ratio 0.75
Plain full search	1.32 (1.00×)	1.32 (1.00×)
Focus rescoring (unmasked)	1.66 (0.80×)	1.64 (0.81×)
Masked, $L=0$	1.29 (1.03×)	1.30 (1.02×)
M2, $L=1$	1.58 (0.84×)	1.97 (0.67×)
M2, $L=3$	1.55 (0.85×)	1.99 (0.67×)
M1	1.57 (0.85×)	1.99 (0.67×)
V1, $L=1, m=8$	1.81 (0.73×)	2.25 (0.59×)

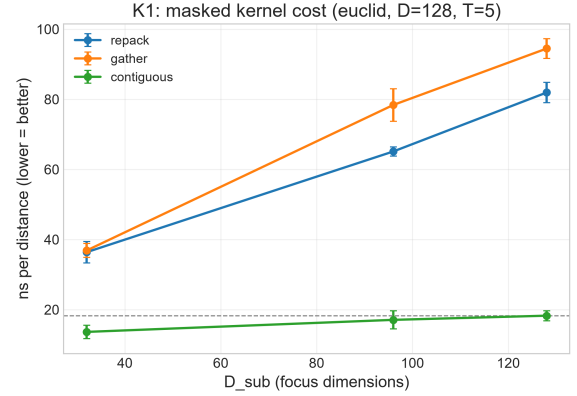


Figure 4: K1: cost of one distance at SIFT width (Euclidean, $D=128$, host B) for D_{sub} focus coordinates, by kernel. The dashed line is one full contiguous 128-d distance, as plain search computes it. Mean±std over $T=5$ trials.

5.6 V1: Verify + Overfetch Recovers Recall Where Coherence Allows

Since masked search returns subspace neighbors, can a post-traversal full-distance pass recover the full-space ones? At over-fetch $m=1$, masked and verified Recall@10 are identical, as predicted in Section 4. At $m=8$ on SIFT1M with $L=1$, full-space recall rises well above the coherence level, from 0.24 to 0.61 at ratio 0.5 and from 0.48 to 0.91 at ratio 0.75 (Figure 5); with every layer masked the results are similar (0.64 and 0.93). The candidate pool is now the top-80 under the focus metric, and verification recovers whatever part of the full-space top-10 it contains; at ratio 0.1 that part is small (0.01 → 0.06). In-engine, overfetching and verifying add 0.23–0.28 ms (Table 3), of which the rescore itself is at most 0.09 ms.

5.7 M3: Blending Never Beats Pure Masking

Finally, we test whether M3’s continuous α can improve on M2. Figure 6 shows that on SIFT1M at ratio 0.25, subspace recall is highest at $\alpha=0$ for every cutoff (0.77 at $L=1$ and 0.78 at $L=2$, matching Table 2) and falls monotonically to the $L=0$ (coherence) level of 0.075 as $\alpha \rightarrow 1$; already at $\alpha=0.25$ it drops to 0.21. Since $\alpha=0$ is exactly M2, blending adds API surface and computes both distances without ever improving on the simpler design; we do not recommend shipping M3.

5.8 Summary

Table 4 consolidates the verdict for each mechanism.

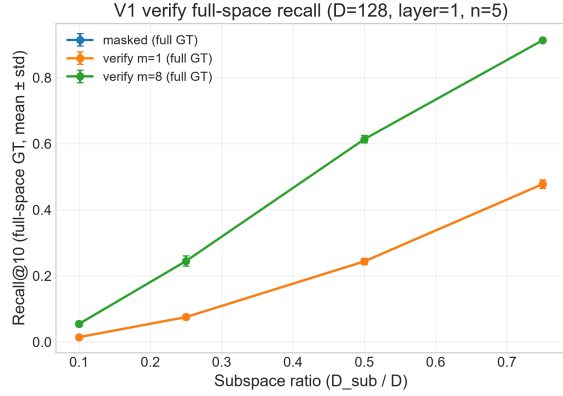


Figure 5: V1 on SIFT1M with $L=1$: full-space Recall@10 without verification, with verification at $m=1$, and at $m=8$ (mean $\pm$ std, $T=5$). The unverified curve is hidden under $m=1$, which returns the same set by construction.

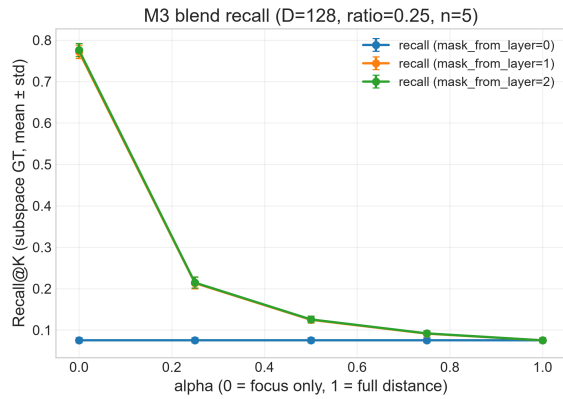


Figure 6: M3 on SIFT1M: subspace Recall@10 vs. α for cut-offs $\{0, 1, 2\}$ at ratio 0.25 (mean $\pm$ std, $T=5$).

Table 4: Verdict per mechanism.

Mechanism	Verdict	Evidence
Attribution (in-DB)	Keep	+0.3 ms in-engine; $\sim 1.8\times$ vs. post-query
Focus rescoring only	Not accel.	$0.80\times$ plain speed (§5.2)
M1 all-layer mask	Keep	Full-space recall = coherence; subspace recall 0.38–0.98 (§5.3)
M2 hybrid cutoff	Optional	At most +0.04 subspace recall over M1 (§5.4)
K1 gather kernel	Skip	No faster than repack in isolation; within 3% in-engine (§5.5)
C1 coherence	Keep	Equals full-space recall of masked search
V1, $m=1$	API only	Recall@ k provably unchanged
V1 + overfetch	Keep	Recovers full-space recall where coherence allows; +0.3 ms
M3 blend	Skip	Best at $\alpha=0$, i.e. M2

6 Related Work

Vector databases. Milvus [25], Pinecone [21], and Qdrant [22] expose ANN with filters and quantization; AnalyticDB-V [28] and PASE [29] bring ANN indexes into relational engines, and VBase [30] unifies vector search with relational operators. Pan et al. [19] survey the design space. None exposes per-coordinate score decompositions or layer-aware masked metrics. We add

in-database attribution and query-time masked HNSW scoring inside an existing production graph index.

ANN search. FAISS [12] and ScaNN [10] optimize large-scale search, and experimental studies compare ANN methods under common protocols [1, 4, 27]; explainability and dynamic subspace routing remain outside the index. ADSampling [6], FINGER [3], and RaBitQ [7] speed up search by making distance comparisons cheaper; our latency results (Section 5.5) suggest masked search needs comparably cheap comparisons to pay off.

Subspace and constrained search. k -NN queries over arbitrary query-time subspaces have been studied with dedicated index structures [2, 15], and MUST [26] searches concatenated multimodal vectors under learned per-modality weights, of which a modality mask is a special case. Filtered graph search faces a related problem: when a predicate changes which points qualify, a graph built for the unconstrained query can lose navigability, which ACORN [20] and Filtered-DiskANN [8] address by changing graph construction. We instead measure what an unmodified full-space HNSW graph delivers under a query-time masked metric.

Explaining query answers. Database work explains query results through causality and intervention over input tuples [18, 24]. A similarity score is instead a sum of per-coordinate terms, the aggregate-of-grades score model of top- k processing [5], so explaining it needs no search over inputs: attribution reports each coordinate’s exact share of the score, as an OLAP roll-up reports each group’s share of a total [9]. Model explainers such as SHAP and LIME [16, 23] estimate attributions by sampling or local surrogates instead.

Dimension reduction and projected indexes. Static projections or multi-index layouts differ from *query-time* focus sets evaluated against an existing full-dimensional HNSW index. A dedicated auxiliary index per focus set would avoid both the recall loss for small focus sets (Section 5.4) and the per-distance cost (Section 5.5), but only for a registered palette of subspaces; we leave it to future work (Section 7).

7 Discussion and Conclusion

Shipping verdict. Keep in-database attribution. Keep masked traversal as a semantics feature for subspace search, with V1 overfetch when callers need full-space neighbors; the layer cutoff can stay as an optional knob, and M3 should go. In this implementation masked traversal is not a speed feature.

Latency. Masking lost to plain search in every configuration because a masked distance costs more than a full contiguous one, whichever kernel computes it (Section 5.5). Laying registered focus sets out contiguously, or a dedicated projected index per registered focus set (Section 6), are the natural next steps.

Limitations. SIFT1M is our only corpus on which plain HNSW search is accurate; on the i.i.d. synthetic corpus plain search reaches only 0.19 Recall@10, and real encoder embeddings remain to be tested. We did not sweep ef for masked queries, each masked-traversal result comes from a single index per corpus, and kernel costs are from one x86 CPU.

XQdrant shows exact feature attribution can live in a Rust vector engine at a small in-engine cost and a clear end-to-end win over client-side extraction. For masked-distance HNSW, the main lesson is methodological: recall must be measured against

the right ground truth. Against full-space ground truth, masked traversal of a full-space graph scores the subspace/full-space coherence, a property of the data rather than of the index; against subspace ground truth, it navigates well for large focus sets and degrades for small ones. It never beats plain search, because each masked distance costs more than a full contiguous one. The public experiment index (Section 8) maps each claim to a reproducible run.

8 Artifacts

Source code (XQdrant fork of Qdrant) and the experiment harness are public:

- Engine fork at the evaluated commit: <https://github.com/Moetez-Fradi/XQdrant/tree/ed5ca79>; the kernel microbenchmark of Figure 4 is commit 6cb79ea, which adds only the benchmark
- Experiment index and scripts: https://github.com/Moetez-Fradi/XAI/tree/EDBT27/DB_systems

The harness contains the unified orchestration script, per-mechanism runners, the in-engine latency checks, plotting entry points for M1–M3, K1, C1, and V1, and build/run notes for the XQdrant binary and the vanilla Qdrant baseline. Neither repository uses access monitoring.

References

- [1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374.
- [2] Thomas Bernecker, Tobias Emrich, Franz Graf, Hans-Peter Kriegel, Peer Kröger, Matthias Renz, Erich Schubert, and Arthur Zimek. 2010. Subspace Similarity Search: Efficient k-NN Queries in Arbitrary Subspaces. In *Proceedings of the International Conference on Scientific and Statistical Database Management (SSDBM)*.
- [3] Patrick H. Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit S. Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference (WWW)*.
- [4] Karima Echihabi, Kostas Zoumpatianos, Themis Palpanas, and Houda Benbrahim. 2019. Return of the Lernaean Hydra: Experimental Evaluation of Data Series Approximate Similarity Search. *Proceedings of the VLDB Endowment* 13, 3 (2019). doi:10.14778/3368289.3368303
- [5] Ronald Fagin, Amnon Lotem, and Moni Naor. 2003. Optimal Aggregation Algorithms for Middleware. *J. Comput. System Sci.* 66, 4 (2003), 614–656. Preliminary version in PODS 2001.
- [6] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023). SIGMOD 2023.
- [7] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 167:1–167:27. doi:10.1145/3654970 SIGMOD 2024.
- [8] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, et al. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference (WWW)*.
- [9] Jim Gray, Surajit Chaudhuri, et al. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery* 1, 1 (1997), 29–53.
- [10] Ruiqi Guo, Philip Sun, Erik Lindgren, et al. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*.
- [11] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572
- [13] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. 2022. Matryoshka Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35.
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33.
- [15] Xiang Lian and Lei Chen. 2008. Similarity Search in Arbitrary Subspaces Under Lp-Norm. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.
- [16] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [17] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [18] Alexandra Meliou, Wolfgang Gatterbauer, Katherine F. Moore, and Dan Suciu. 2010. The Complexity of Causality and Responsibility for Query Answers and Non-Answers. *Proceedings of the VLDB Endowment* 4, 1 (2010), 34–45.
- [19] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of Vector Database Management Systems. *The VLDB Journal* 33 (2024).
- [20] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024). SIGMOD 2024.
- [21] Pinecone Systems. 2023. Pinecone: Managed Vector Database for Production AI. <https://www.pinecone.io/>.
- [22] Qdrant Team. 2024. Qdrant: Vector Search Engine Documentation. <https://qdrant.tech/documentation/>.
- [23] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. doi:10.1145/2939672.2939778
- [24] Sudeepa Roy and Dan Suciu. 2014. A Formal Approach to Finding Explanations for Database Queries. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1579–1590. doi:10.1145/2588555.2588578
- [25] Jianguo Wang, Xiaomeng Yi, Rentong Guo, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. doi:10.1145/3448016.3457550
- [26] Mengzhao Wang, Xiangyu Ke, Xiaoliang Xu, Lu Chen, Yunjun Gao, Pinpin Huang, and Runkai Zhu. 2024. MUST: An Effective and Scalable Framework for Multimodal Search of Target Modality. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*.
- [27] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978.
- [28] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165. doi:10.14778/3415478.3415541
- [29] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 2241–2253. doi:10.1145/3318464.3386131
- [30] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.